LOW-CODE EXPERIMENT

Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Student Name	Thanashree G R
Roll Number	241501262
Department	Artificial Intelligence and Machine Learning (AIML)
Application	Patient Appointment Booking System

Abstract

This case study examines how a low-code application such as Zoho Creator may face challenges when a successful small-scale application is expanded into a complex, multi-branch enterprise system. The Patient Appointment Booking System is used as the scenario. The study considers performance, simultaneous bookings, advanced analytics, integrations, data volume, customization, security governance, and maintainability. A hybrid architecture is recommended: retain low-code for forms, approvals, workflows, and dashboards while using dedicated services for high-load and computation-intensive functions.

1. Aim

To study the limitations of low-code platforms in a complex enterprise use case and recommend a suitable architecture for scaling the Patient Appointment Booking System.

2. Introduction

Low-code platforms speed up application development through visual form builders, reports, workflows, and managed hosting. They work well for many workflow-centred applications. However, as users, records, integrations, and transactions increase, limits may appear. This experiment identifies those limits and explains how low-code can be combined with pro-code services.

3. Case Scenario

A fictional healthcare group operates 250 clinics across several cities. Its Patient Appointment Booking System was first built in Zoho Creator for a single clinic. Management now wants to deploy it group-wide. The expanded system must:

- Synchronize doctor availability and appointment slots across clinics in near real time.
- Handle thousands of simultaneous booking requests without double-booking a doctor or time slot.
- Forecast appointment demand using machine learning and recommend staffing levels.
- Integrate with hospital records, billing, payment, loyalty, insurance, and messaging systems.
- Protect sensitive patient information and maintain consistent access controls and audit trails.
- Generate reports across millions of appointment records and multiple branches.

4. Analysis of Low-Code Limitations

Area	Potential limitation	Impact on the appointment system
Performance and scalability	Execution-time, API-call, and resource limits may restrict heavy processing.	Busy booking periods and large reports may become slow or fail.
Concurrency and transactions	Fine-grained transaction and locking controls may be limited.	Simultaneous users may try to reserve the same doctor and time slot.
Advanced algorithms	Business-rule scripting is not ideal for intensive machine-learning calculations.	Demand forecasting and optimization may need a separate ML service.
Integration depth	Connectors and REST APIs cover common cases, but complex legacy protocols may need middleware.	Connecting hospital, billing, insurance, and messaging systems becomes harder to maintain.
Data volume	Reports may not be designed for large-scale analytical workloads.	Cross-branch analytics may be slow; a data warehouse may be needed.
Customization	Visual builders have boundaries on layout and behavior.	Specialized patient journeys may require custom components.
Vendor lock-in	Workflows and data models may depend on platform-specific features.	Migration may require significant redevelopment.
Governance and security	Many apps and makers can create inconsistent permissions and practices.	Patient privacy and access management need central control.
Testing and DevOps	Automated testing, version control, and deployment pipelines may need extra tools.	Changes across many branches are harder to validate and release safely.

Most critical risks: concurrency and performance. Appointment booking must prevent conflicting reservations, and high-demand periods can create many simultaneous requests. Machine-learning forecasts and large-scale analytics are also better handled by specialized services.

5. Mitigation Strategies

• **Hybrid architecture:** Keep forms, approvals, routine workflows, and dashboards in Zoho Creator; move high-load booking logic and complex calculations to dedicated services.
• **Prevent conflicting bookings:** Confirm slot availability atomically through a backend service and transactional database controls.
• **Integration layer:** Use documented APIs or an API gateway to connect hospital records, billing, insurance, and messaging systems.
• **Separate analytics:** Use a data warehouse or analytics platform for large cross-clinic reports and forecasting.
• **Governance:** Apply role-based, least-privilege access, audit trails, naming standards, and controlled development/testing/production releases.
• **Testing:** Conduct proof-of-concept, load, security, backup, and recovery tests before rollout.
• **Reduce vendor lock-in:** Keep data exportable, document business rules and APIs, and maintain a migration plan.

6. Recommended Architecture

Component	Recommended approach
Patient forms, routine workflows, approvals and staff dashboards	Zoho Creator low-code application
Real-time appointment-slot reservation	Dedicated backend service with transactional database controls, exposed through an API
Demand forecasting and staffing recommendations	Separate analytics or machine-learning service
Cross-branch reporting	Data warehouse and business intelligence tool
Hospital and third-party integrations	Integration layer or API gateway
Security and auditing	Centralized identity, role-based access, audit logging, and periodic review

7. Recommended Decision

A complete enterprise appointment engine should not rely on low-code alone when strict concurrency, high throughput, advanced analytics, and complex integrations are essential. Keep Zoho Creator for patient data entry, routine workflows, approvals, and reports. Use a dedicated backend for critical slot reservation and transaction control, plus separate services for machine learning and large-scale analytics.

8. Conclusion

Low-code platforms are valuable for developing workflow-centred applications quickly, but they are not a universal solution. Scaling the Patient Appointment Booking System introduces challenges in performance, simultaneous bookings, analytics, integrations, security governance, and deployment. A hybrid architecture preserves low-code productivity while assigning high-load and specialized responsibilities to suitable services.

9. Learning Outcome

This case study develops the ability to assess whether a low-code platform suits a business requirement, recognize technical and organizational limitations at enterprise scale, and propose a practical hybrid architecture.

Result: The limitations of low-code platforms were studied, and a hybrid architecture was recommended for scaling the Patient Appointment Booking System reliably.

Reference: Low-code experiment case-study structure; scenario adapted to the Patient Appointment Booking System.